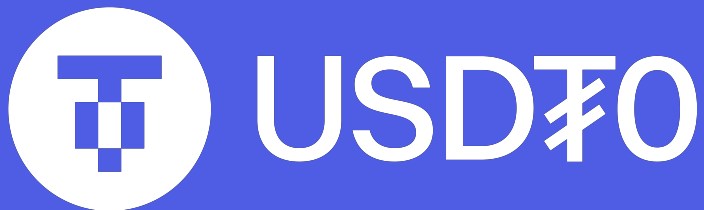


Hedera Token Connector Audit



July 29, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Low Severity	7
L-01 Missing Validation of Treasury Ownership and Supply Key in HTS Connector	7
L-02 Unused Contract	7
L-03 Floating Pragma	8
Notes & Additional Information	8
N-01 Payable initialize Function Risks Locking Native Tokens	8
N-02 Incorrect Casting of HTS Precompile Return Values	9
N-03 Unused Functionalities in HederaTokenService	10
N-04 Use Custom Errors	10
N-05 License Requires Attribution	11
N-06 Missing Zero-Address Check	11
N-07 Duplicate Import	12
Conclusion	13

Summary

Type	Stablecoins	Total Issues	10 (5 resolved)
Timeline	From 2025-07-14 To 2025-07-18	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	3 (1 resolved)
		Notes & Additional Information	7 (4 resolved)

Scope

OpenZeppelin audited [pull request #106](#) of the [Everdawn-Labs/usdt0-oft-contracts](#) repository at commit [4a03610](#).

In scope were the following files:

```
contracts
├── hts
│   ├── HederaTokenService.sol
│   ├── IHederaTokenService.sol
│   └── KeyHelper.sol
└── OUpgradeableHTSConnectorExistingToken.sol
```

System Overview

[Pull request #106](#) contains the necessary files to allow a native token on the Hedera network to be connected as an Omnichain Fungible Token (OFT) through LayerZero integration. This is done by having the `0UpgradeableHTSConnectorExistingToken` contract act as the treasury account for the native token through the [Hedera Token Service](#) (HTS), which is the Hedera network's native service for creating and managing tokens of various types. In addition, the connector contract must also be granted Supply Key privileges in order to mint and burn tokens.

Cross-chain calls through LayerZero are directed to the `0UpgradeableHTSConnectorExistingToken` contract which, in turn, connects with the native token through the HTS. The HTS connector contract is then responsible for minting tokens to the set treasury account on the Hedera network. These tokens are then transferred to the recipient when required. In the case of token burning, the HTS connector contract transfers tokens from the spender account to the connector contract, from which it is then burned. It should be noted that the native USDT0 token has not been deployed on Hedera at the time of this review.

Security Model and Trust Assumptions

During the review, the following observations and trust assumptions were made:

- The proxy contract will be deployed and initialized within the same transaction to prevent the front-running of the `initialize` function of the `0UpgradeableHTSConnectorExistingToken` contract.
- The USDT0 token will have the same number of decimals (6) on all chains.
- No custom fees will be enabled for the USDT0 token.
- The proxy delegating to the `0UpgradeableHTSConnectorExistingToken` contract will be set as the treasury account for the native token on the Hedera network.

- The Supply Key used to grant the HTS connector contract mint and burn permissions will be a native-threshold keylist. It will have a threshold of 1/2, with the HTS connector contract being one party and the Everdawn multisig on the Hedera network being the other.

Low Severity

L-01 Missing Validation of Treasury Ownership and Supply Key in HTS Connector

The `0UpgradeableHTSConnectorExistingToken` contract assumes that it is the treasury account for any HTS token provided during deployment. This assumption is critical for the correct functioning of the connector but is never enforced. In addition, the connector requires the Supply Key of the HTS token in order to successfully execute the `mintToken` and `burnToken` precompile calls within the `_credit` and `_debit` functions. However, this requirement is also not validated.

If the connector is not the actual treasury of the token, the `_credit` and `_debit` core functions will fail in ways that can cause permanent loss of funds. In `_credit`, tokens are minted to the configured treasury account, but the connector fails to transfer them due to an `INSUFFICIENT_TOKEN_BALANCE` error. In `_debit`, the connector pulls tokens from the user but attempts to burn them from the treasury account, leaving the user's tokens stranded in the connector. Furthermore, without the Supply Key, minting and burning operations will fail entirely, effectively bricking the bridge for that token.

Consider adding validations to ensure that the connector is the treasury of the given HTS token and holds the required Supply Key permissions. These checks should occur during initialization to prevent misconfiguration and ensure that the connector can reliably execute mint and burn operations. Alternatively, if such validation is not implemented, consider clearly documenting the limitations to ensure that integrators understand that the connector only functions correctly when it is the designated treasury and has the appropriate key permissions.

Update: Acknowledged, will resolve. The Everdawn team stated:

| We will verify this with config-verification.

L-02 Unused Contract

The `0UpgradeableHTSConnectorExistingToken` contract inherits the `KeyHelper` abstract contract but fails to utilize any of its functions or properties. This inheritance introduces unnecessary complexity to the contract's architecture, potentially obscuring the

contract's actual dependencies and intended functionality. The presence of unused inherited contracts can mislead developers and auditors about the contract's behavior and interactions.

Consider removing the inheritance of `KeyHelper` from `0UpgradeableHTSConnectorExistingToken` if its functionalities are deemed unnecessary. This action will streamline the contract, reducing its complexity and the associated costs of deployment.

Update: Resolved in [pull request #108](#) at [commit e7729b6](#).

L-03 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

The `0UpgradeableHTSConnectorExistingToken` contract has the `solidity ^0.8.20` floating pragma directive.

Consider using fixed pragma directives.

Update: Acknowledged, not resolved. The Everdawn team stated:

We have increased the pragma directive to `^0.8.22` so it matches the pragma directive of the `0Upgradeable` contract.

Notes & Additional Information

N-01 Payable `initialize` Function Risks Locking Native Tokens

The `initialize` function of the `0UpgradeableHTSConnectorExistingToken` contract is declared as `payable`. This allows it to accept native tokens (e.g., HBAR) during the contract's initialization phase. However, without a dedicated mechanism to withdraw these native tokens or a fallback function designed to handle such transfers, any native tokens sent

to the contract upon initialization will be permanently locked. This could lead to the potential loss of native assets.

To prevent the accidental or deliberate transfer of native tokens to the contract during its initialization, consider removing the `payable` modifier from the `initialize` function.

Update: Resolved in [pull request #108](#) at [commit eddf9d5](#).

N-02 Incorrect Casting of HTS Precompile Return Values

The `updateTokenKeys` function of the `HederaTokenService` contract casts the return value of the HTS precompile call to `int32`, even though, according to [HIP-514](#), this function actually returns an `int64` value. Similarly, other HTS precompile functions such as `mintToken`, `burnToken`, and `transferToken` are cast to `int32` despite returning an `int` value as specified by [HIP-206](#). These casts introduce type mismatches between the actual return values and the declared types in the contract.

Improper casting of wider return types to `int32` can lead to truncation or loss of data if a return value exceeds the 32-bit range. This may result in incorrect response code interpretation, especially if future Hedera updates introduce new response codes outside the `int32` range. These silent type mismatches can cause unexpected behavior in error handling and access control logic, reduce code robustness, and break alignment with the Hedera precompile interface specifications.

Consider updating the `HederaTokenService` contract to match the return types with those defined in [HIP-514](#) and [HIP-206](#). Specifically, use `int64` for the return value of `updateTokenKeys` and use `int` for `mintToken`, `burnToken`, and `transferToken`. Maintaining strict type alignment with the Hedera interface improves clarity, prevents silent truncation issues, and ensures compatibility with future platform updates.

Update: Acknowledged, will resolve. The Everdawn team stated:

We will create a ticket to resolve the return value type issue. However, the engineering team has confirmed that the returned values fit well within the limits of an `int32` container.

N-03 Unused Functionalities in HederaTokenService

The `HederaTokenService` contract includes unused definitions such as the `createFungibleToken` function and the `CallResponseEvent` event, neither of which is utilized within the current scope of the HTS connector implementation. The `HederaTokenService` contract was imported from the official [Hashgraph repository](#) and includes a broad set of HTS precompile interfaces, many of which are irrelevant to the HTS connector's functionality.

Leaving unused functions and events in the contract reduces code clarity, increases the cognitive load for auditors and maintainers, and contributes to higher deployment costs due to unnecessary bytecode. In addition, unused functionalities like `CallResponseEvent` and `createFungibleToken` may give the false impression that certain features are implemented or supported by the connector when they are not, potentially leading to confusion during integration.

Consider importing and using the full, unedited version of the `HederaTokenService` contract to improve consistency, readability, and auditability. Alternatively, consider extracting and retaining only the minimal set of functionalities that are actually used by the HTS connector, removing unused functionalities to reduce contract size and deployment cost.

Update: Resolved in [pull request #108](#) at [commit 7320711](#).

N-04 Use Custom Errors

Since Solidity version 0.8.4, custom errors provide a cleaner and more cost-efficient way to explain to users why an operation failed.

In the `OUpgradeableHTSConnectorExistingToken` contract, multiple instances of `revert` and/or `require` messages were identified:

- The `require(_amountLD <= uint64(type(int64).max), "HTSConnector: amount exceeds int64 safe range")` statement
- The `require(transferResponse == HederaTokenService.SUCCESS_CODE, "HTS: Transfer failed")` statement
- The `require(response == HederaTokenService.SUCCESS_CODE, "HTS: Burn failed")` statement
- The `require(_amountLD <= uint64(type(int64).max), "HTS: amount exceeds int64 safe range")` statement

- The `require(response == HederaTokenService.SUCCESS_CODE, "HTS: Mint failed")` statement
- The `require(transferResponse == HederaTokenService.SUCCESS_CODE, "HTS: Transfer failed")` statement

For conciseness and gas savings, consider replacing `require` and `revert` messages with custom errors.

Update: Resolved in [pull request #108](#) at [commit 5d09732](#).

N-05 License Requires Attribution

The `HederaTokenService` contract used is a modified version of the `HederaTokenService` contract from the official Hashgraph repository. The original contract uses the [Apache 2.0 license](#), which requires attribution in derived works.

Consider adding the appropriate attributions within the modified `HederaTokenService` contract.

Update: Acknowledged, will resolve. The Everdawn team stated:

| *We have started working on the fix.*

N-06 Missing Zero-Address Check

When operations with address parameters are performed, it is crucial to ensure the address is not set to zero. Setting an address to zero is problematic because it has special burn/renounce semantics. This action should be handled by a separate function to prevent accidental loss of access during value or ownership transfers.

Within the `constructor` of the `0UpgradeableHTSConnectorExistingToken` contract, the `_htsTokenAddress` is missing a zero-address check.

Consider always performing a zero-address check before performing any important operations or assigning a state variable.

Update: Acknowledged, not resolved.

N-07 Duplicate Import

In `0UpgradeableHTSConnectorExistingToken.sol`, both `HederaTokenService.sol` and `IHederaTokenService.sol` are imported directly. However, `HederaTokenService.sol` already `imports` `IHederaTokenService.sol` internally. This results in a redundant and unnecessary duplicate import of the same interface.

Consider removing duplicate imports to improve the overall clarity and maintainability of the codebase.

Update: Resolved in [pull request #108](#) at [commit 4bd2d4f](#).

Conclusion

The reviewed USDT0 contracts enable the use of LayerZero OFT functionality on the Hedera network through a Hedera Token Service (HTS) connector contract. The HTS contract enables cross-chain minting and burning through integrating with the native HTS service. Overall, the codebase was found to be well implemented, and only low-severity issues and informational notes were reported. The Everdawn team was responsive throughout the audit, providing valuable information regarding the in-scope codebase whenever requested.